

PAGE 1 — Python Core Essentials for AI/ML

1 Syntax & Core Constructs (Ultra-Fast Recap)

Variables & Dynamic Typing

```
x = 10
x = "now string"    # dynamic typing
type(x)

a: int = 5           # type hint (not enforced at runtime)
b: float | None = 3.2
```

Multiple Assignment

```
a, b = 1, 2
a, b = b, a          # swap
x = y = 0            # chained
a, *rest = [1,2,3,4] # unpacking
```

f-Strings

```
name = "AI"
score = 0.98765
f"{name=}, {score:.3f}"
```

Ternary

```
label = "pos" if y > 0 else "neg"
```

Walrus :=

```
if (n := len(data)) > 100:
    print(n)
```

Truthy / Falsy

Falsy:

```
False, None, 0, 0.0, "", [], {}, set()
```

None vs False

```
if x is None: # identity check
if not x:     # falsy check (0, "", etc.)
```

Type Hints (Minimal Useful Set)

```
from typing import List, Dict, Tuple, Optional, Union, Callable

def f(x: int, y: float) -> float:
    return x + y
```

2 Data Structures (Performance-Oriented) List

```
lst = [1,2,3]
lst.append(4)
lst.extend([5,6])
lst.pop()
lst.sort(reverse=True)
```

Tuple (immutable, hashable)

```
t = (1,2,3)
d = {(1,2): "point"} # tuple as dict key
```

Use tuple when:

- Fixed structure
- Dictionary key
- Want immutability

Set

```
s = {1,2,3}
s.add(4)
s.remove(2)
s1 & s2    # intersection
s1 | s2    # union
```

Dict

```
d = {"a":1, "b":2}
d["c"] = 3
d.get("x", 0)
d.keys(); d.values(); d.items()
```

Dict Merge (3.9+)

```
d3 = d1 | d2
```

Comprehensions (Essential)

```
[x*x for x in range(10)]
{x: x*x for x in range(5)}
{x for x in range(10) if x%2==0}
```

Nested:

```
[(i,j) for i in range(3) for j in range(3)]
```

Conditional:

```
[x if x%2==0 else 0 for x in range(10)]
```

collections (AI-Friendly Tools)

```
from collections import defaultdict, Counter, deque, namedtuple
```

defaultdict

```
dd = defaultdict(int)
dd["a"] += 1
```

Counter

```
c = Counter(["a", "b", "a"])
c.most_common(1)
```

deque (fast queue)

```
dq = deque([1,2])
dq.appendleft(0)
dq.pop()
```

namedtuple

```
Point = namedtuple("Point", "x y")
p = Point(1,2)
p.x
```

3 Functions

Basic

```
def f(x, y=0):
    return x + y
```

*args, **kwargs

```
def f(*args, **kwargs):
    print(args)
    print(kwargs)
```

```
f(1,2,a=3)
```

Keyword-Only Args

```
def f(x, *, lr=0.01):
    ...
```

Default Mutable Trap

```
def bad(lst=[]): # DON'T
    lst.append(1)
```

```
def good(lst=None):
    if lst is None:
        lst = []
```

Lambda

```
f = lambda x: x**2
sorted(data, key=lambda x: x[1])
```

Closures

```
def outer(a):
    def inner(b):
        return a + b
    return inner
```

```
add5 = outer(5)
add5(3)
```

Function Annotations

```
def train(X: list[float]) -> float:
    ...
```

Minimal Docstring

```
def f(x):
    """Compute squared value."""
    return x*x
```

4 Iteration & Functional Patterns

enumerate

```
for i, val in enumerate(data):
    ...
```

zip

```
for x, y in zip(xs, ys):
    ...
```

map / filter

```
list(map(str, nums))
list(filter(lambda x: x>0, nums))
```

any / all

```
any(x > 0 for x in nums)
all(x > 0 for x in nums)
```

sorted with key

```
sorted(data, key=len)
sorted(data, key=lambda x: x[1])
```

Custom multi-key:

```
sorted(data, key=lambda x: (x[1], -x[2]))
```

itertools (Power Tools)

```
import itertools as it
```

Cartesian product:

```
list(it.product([1,2], [3,4]))
```

Permutations:

```
list(it.permutations([1,2,3], 2))
```

Combinations:

```
list(it.combinations([1,2,3], 2))
```

Chain:

```
list(it.chain([1,2],[3,4]))
```

Cycle:

```
c = it.cycle([1,2])
next(c)
```

Islice:

```
list(it.islice(range(100), 5))
```

5 Exception Handling

Basic Pattern

```
try:
    risky()
except ValueError as e:
    print(e)
else:
    print("success")
finally:
    cleanup()
```

Multiple:

```
except (TypeError, KeyError):
```

Custom Exception

```
class DataError(Exception):
    pass

raise DataError("bad dataset")
```

Re-raise:

```
except Exception:
    raise
```

6 File IO (Data Pipeline Basics)

Read Text

```
with open("file.txt") as f:
    data = f.read()
```

Line-by-line:

```
for line in open("file.txt"):
    ...
```

Write:

```
with open("out.txt","w") as f:
    f.write("hello")
```

JSON

```
import json

with open("data.json") as f:
    obj = json.load(f)

json.dumps(obj)
```

CSV

```
import csv

with open("data.csv") as f:
    reader = csv.reader(f)
    for row in reader:
        ...
```

pathlib (modern path handling)

```
from pathlib import Path

p = Path("data") / "file.txt"
p.exists()
p.read_text()
```

7 Virtual Environments (ML Hygiene)

Create venv

```
python -m venv .venv
source .venv/bin/activate
```

Install

```
pip install numpy pandas scikit-learn
```

Freeze

```
pip freeze > requirements.txt  
pip install -r requirements.txt
```

Minimal pyproject.toml

```
[project]  
name = "ml-project"  
version = "0.1.0"  
dependencies = ["numpy", "pandas"]
```



PAGE 1 MENTAL MODEL

If you master this page:

- You can write clean ML scripts
 - Build data pipelines
 - Avoid common Python bugs
 - Manipulate data efficiently
 - Structure ML projects properly
-

PAGE 2 — Advanced Python for AI/ML

1 OOP (Only What Matters for ML Code)

Basic Class

```
class Model:
    def __init__(self, lr: float):
        self.lr = lr
        self.weights = None

    def fit(self, X, y):
        ...

    def predict(self, X):
        ...
```

__repr__ (Debugging Essential)

```
def __repr__(self):
    return f"{self.__class__.__name__}(lr={self.lr})"
```

Instance vs Class Variables

```
class A:
    count = 0          # class var

    def __init__(self):
        A.count += 1   # shared
```

@staticmethod

No access to instance.

```
class Utils:
    @staticmethod
    def normalize(x):
        return (x - min(x)) / (max(x) - min(x))
```

@classmethod

Access class.

```
class Model:
    def __init__(self, lr):
        self.lr = lr

    @classmethod
    def from_config(cls, cfg: dict):
        return cls(lr=cfg["lr"])
```

Common Dunder Methods

```
__len__
__getitem__
__setitem__
__call__
```

```
__iter__
__add__
__eq__
```

Example:

```
class Dataset:
    def __init__(self, data):
        self.data = data

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx]
```

Callable object:

```
class Linear:
    def __call__(self, x):
        return x * 2
```

@dataclass (Clean Config Objects)

```
from dataclasses import dataclass

@dataclass
class Config:
    lr: float
    epochs: int = 10
```

With defaults:

```
@dataclass(frozen=True)
class Params:
    dim: int
```

Composition > Inheritance

Prefer:

```
class Trainer:
    def __init__(self, model):
        self.model = model
```

Avoid deep inheritance chains.

2 Modules & Packaging

Project Structure (ML Template)

```
project/
├── src/
│   ├── __init__.py
│   ├── data.py
│   ├── model.py
│   └── train.py
├── tests/
└── pyproject.toml
```

└─ README.md

Absolute Import (Preferred)

```
from src.model import Model
```

Relative Import (Inside Package)

```
from .model import Model
```

__init__.py

Expose clean API:

```
from .model import Model
from .data import load_data
```

Editable Install (Dev Mode)

```
pip install -e .
```

Now import anywhere:

```
from project.model import Model
```

3 Performance

List vs Generator

List:

```
[x*x for x in range(10_000_000)]
```

Generator:

```
(x*x for x in range(10_000_000))
```

Memory difference:

- List → stores everything
- Generator → lazy

Timing

time

```
import time
t0 = time.time()
run()
print(time.time() - t0)
```

timeit

```
import timeit
timeit.timeit("sum(range(1000))", number=1000)
```

cProfile

```
python -m cProfile train.py
```

Inside:

```
import cProfile
cProfile.run("train()")
```

Memory Profiling (Quick Trick)

```
import sys
sys.getsizeof(obj)
```

Better:

```
pip install memory-profiler
```

Big-O Quick Intuition

Operation	Complexity
list append	$O(1)$
dict lookup	$O(1)$ avg
sorting	$O(n \log n)$
nested loops	$O(n^2)$

Avoid:

```
for x in list:
    if x in list: #  $O(n^2)$ 
```

Use set:

```
s = set(list)
```

4 Decorators (Power Feature)

Basic Pattern

```
def decorator(func):
    def wrapper(*args, **kwargs):
        print("Before")
        result = func(*args, **kwargs)
        print("After")
        return result
    return wrapper
```

Usage:

```
@decorator
def train():
    ...
```

Timing Decorator

```
import time

def timer(func):
    def wrapper(*args, **kwargs):
        t0 = time.time()
        out = func(*args, **kwargs)
        print(time.time() - t0)
        return out
    return wrapper
```

Decorator with Arguments

```
def repeat(n):
    def deco(func):
        def wrapper(*a, **kw):
            for _ in range(n):
                result = func(*a, **kw)
            return result
        return wrapper
    return deco

@repeat(3)
def f():
    print("hi")
```

Preserve Metadata

```
from functools import wraps

def deco(func):
    @wraps(func)
    def wrapper(*a, **kw):
        return func(*a, **kw)
    return wrapper
```

5 Typing (Useful Subset for ML)

```
from typing import List, Dict, Tuple, Union, Optional,
Callable
```

Basic

```
def f(x: List[int]) -> Dict[str, float]:
    ...
```

Union

```
def f(x: int | float) -> float:
    ...
```

Optional

```
def load(path: Optional[str] = None):
```

Callable

```
def train(loss_fn: Callable[[float, float], float]):
```

TypedDict (Structured Dict)

```
from typing import TypedDict

class Batch(TypedDict):
    x: list[float]
    y: list[int]
```

Protocol (Duck Typing Contracts)

```
from typing import Protocol

class ModelLike(Protocol):
    def predict(self, X) -> list: ...
```

Accepts any object implementing predict.

⚠ Type hints:

- Not runtime enforced
- Use mypy for checking

6 Async (Minimal, Practical)

Use when:

- API calls
- Multiple inference requests
- IO-bound workloads

Basic

```
import asyncio

async def fetch():
    await asyncio.sleep(1)
    return 42

async def main():
    result = await fetch()
    print(result)

asyncio.run(main())
```

Parallel Tasks

```
async def main():
    tasks = [fetch() for _ in range(5)]
    results = await asyncio.gather(*tasks)
```

Blocking inside async = BAD:

```
time.sleep(1) # blocks event loop
```

7 Numpy Mental Model (Critical for ML)

Why Arrays > Python Loops

Slow:

```
result = []
for x in xs:
    result.append(x * 2)
```

Fast (vectorized):

```
import numpy as np
xs = np.array(xs)
result = xs * 2
```

Reason:

- C backend
- SIMD
- No Python loop overhead

Broadcasting Intuition

Shapes:

```
(5,3) + (3,) → works
(5,3) + (5,) → error
```

Rule:

- Compare from right
- Equal OR one is 1

Example:

```
X = np.random.randn(5,3)
b = np.random.randn(3)
X + b
```

Axis Logic

2D:

```
axis=0 → down rows
axis=1 → across columns
X.mean(axis=0) # per column
```

Vectorization Mindset

Replace:

```
for i in range(n):
    y[i] = a*x[i] + b
```

With:

```
y = a*x + b
```

Avoid Python loops in ML core math.

If you master this page:

- You write clean ML modules
 - You profile bottlenecks
 - You structure reusable packages
 - You use decorators for logging/timing
 - You understand vectorization
 - You prepare for Numpy / PyTorch / Sklearn internals
-



PAGE 3 — NumPy & Numerical Computing

```
import numpy as np
```

1 ndarray BASICS

Creation

From Python Objects

```
a = np.array([1,2,3])
b = np.array([[1,2],[3,4]])
```

Specify dtype:

```
np.array([1,2,3], dtype=np.float32)
```

Zeros / Ones

```
np.zeros((3,4))
np.ones((2,2))
np.eye(3)           # identity
```

arange (step-based)

```
np.arange(0,10,2)
```

linspace (count-based)

```
np.linspace(0,1,5)
```

Random (Quick)

```
np.random.rand(3,4)           # uniform [0,1)
np.random.randn(3,4)          # normal(0,1)
np.random.randint(0,10,(2,3))
```

Shape & Structure

```
a.shape
a.ndim
a.size
a.dtype
```

Reshape

```
a = np.arange(12)
```

```
a.reshape(3,4)
a.reshape(-1,2)      # infer dimension
```

⚠ Must preserve total size.

Flatten

```
a.flatten()           # copy
a.ravel()              # view (if possible)
```

Transpose

```
A.T
np.transpose(A)
A.swapaxes(0,1)
```

For 3D:

```
A.transpose(0,2,1)
```

2 Indexing (CRITICAL FOR ML)

Basic Slicing

```
a = np.arange(10)
```

```
a[0]
a[2:5]
a[:3]
a[-1]
a[::-2]
```

2D:

```
A[0,1]
A[:,0]
A[1,:]
A[:,2:]
```

Boolean Masking

```
a = np.array([1,2,3,4])
mask = a > 2
a[mask]
```

Inline:

```
a[a % 2 == 0]
```

Modify:

```
a[a < 0] = 0
```

Fancy Indexing

```
a = np.array([10,20,30,40])
a[[0,3,1]]
```

2D:

```
A[[0,2], [1,3]] # pairwise
```

Select rows:

```
A[[0,2], :]
```

Views vs Copies (VERY IMPORTANT)

Slice → view:

```
b = a[2:5]
b[0] = 100 # modifies original
```

Fancy → copy:

```
b = a[[1,3]]
```

Check:

```
b.base is a
```

3 Vectorized Operations

Element-wise

```
a + b
a - b
a * b
a / b
a ** 2
np.sqrt(a)
np.exp(a)
np.log(a)
```

Comparison:

```
a > 0
```

Broadcasting Rules (Essential)

Rule:

- Align from right
- Dimensions equal OR one is 1

Example:

```
X = np.random.randn(5,3)
b = np.random.randn(3)
```

```
X + b
```

Column vector:

```
b = np.random.randn(5,1)
X + b
```

Add scalar:

```
X + 10
```

Force reshape:

```
b.reshape(1,-1)
```

Matrix Multiplication

```
A @ B
np.matmul(A,B)
```

Dot product (1D):

```
np.dot(a,b)
a @ b
```

Batch matmul (3D):

```
np.matmul(A,B)
```

einsum (Advanced Weapon)

Dot:

```
np.einsum("i,i->", a, b)
```

Matrix multiply:

```
np.einsum("ij,jk->ik", A, B)
```

Batch:

```
np.einsum("bij,bjk->bik", A, B)
```

Sum axis:

```
np.einsum("ij->i", A)
```

4 Aggregations

Basic

```
a.sum()
a.mean()
a.std()
a.min()
```

```
a.max()
```

Axis Logic

Given:

```
X.shape = (5,3)
X.sum(axis=0) # column-wise
X.sum(axis=1) # row-wise
```

keepdims

```
X.mean(axis=1, keepdims=True)
```

Keeps shape for broadcasting.

Cumulative

```
np.cumsum(a)
np.cumprod(a)
```

Arg Ops

```
np.argmax(a)
np.argmin(a)
np.argsort(a)
```

5 Random (ML Critical)

Set Seed

```
np.random.seed(42)
```

⚠ Global state (legacy API)

Modern:

```
rng = np.random.default_rng(42)
rng.normal(size=(3,3))
```

Distributions

```
np.random.normal(0,1,(3,3))
np.random.uniform(0,1,(3,3))
np.random.binomial(10,0.5,100)
```

Sampling

```
np.random.choice(10, size=5)
np.random.choice(a, size=3, replace=False)
```

Shuffle:

```
np.random.shuffle(a)
```

6 Linear Algebra Essentials

```
from numpy import linalg as LA
```

Dot Product

```
a @ b
np.dot(a,b)
```

Norm

```
LA.norm(a)
LA.norm(a, ord=1)
```

Inverse

```
LA.inv(A)
```

⚠ Prefer solve:

```
LA.solve(A,b)
```

Determinant

```
LA.det(A)
```

Eigenvalues

```
vals, vecs = LA.eig(A)
```

Symmetric matrix:

```
LA.eigh(A)
```

SVD (VERY IMPORTANT IN ML)

```
U, S, Vt = LA.svd(A)
```

Reconstruct:

```
U @ np.diag(S) @ Vt
```

Pseudo-inverse

```
LA.pinv(A)
```



Performance Micro-Rules

✓ Use vectorization ✓ Avoid Python loops ✓ Prefer @ over manual loops ✓ Use broadcasting ✓ Use float32 for deep learning ✓ Avoid repeated reshapes inside loop

Bad:

```
for i in range(n):  
    X = X.reshape(...)
```



Mental Model Summary

NumPy =

- Homogeneous typed arrays
- C-level speed
- Vectorized computation
- Broadcasting for free parallelism
- Linear algebra backbone of ML

Master this → You unlock:

- Scikit-learn internals
- PyTorch tensor intuition
- Gradient math
- Efficient preprocessing

PAGE 4 — Pandas + Data Handling

```
import pandas as pd
import numpy as np
```

1 DataFrame Basics

Creation From Dict

```
df = pd.DataFrame({
    "age": [20, 25, 30],
    "income": [1000, 2000, 3000]
})
```

From List of Dict

```
data = [{"a": 1, "b": 2}, {"a": 3, "b": 4}]
df = pd.DataFrame(data)
```

From CSV

```
df = pd.read_csv("data.csv")
```

Common params:

```
pd.read_csv("data.csv",
            index_col=0,
            parse_dates=["date"],
            dtype={"age": "int32"},
            na_values=["NA", "?"])
```

Write:

```
df.to_csv("out.csv", index=False)
```

Inspect

```
df.head()
df.tail()
df.shape
df.columns
df.dtypes
df.info()
df.describe()
```

Column Selection

```
df["age"] # Series
```

```
df[["age", "income"]] # DataFrame
```

Assign:

```
df["new"] = df["age"] * 2
```

Row Selection .iloc (position)

```
df.iloc[0]
df.iloc[0:3]
df.iloc[:, 0]
df.iloc[0, 1]
```

.loc (label)

```
df.loc[0]
df.loc[0:2]
df.loc[:, "age"]
df.loc[df["age"] > 20]
```

2 Cleaning

Missing Values

```
df.isna()
df.isna().sum()
df.notna()
```

fillna

```
df["age"].fillna(0)
df.fillna({"age": 0, "income": df["income"].mean()})
df.fillna(method="ffill")
```

dropna

```
df.dropna()
df.dropna(axis=1)
df.dropna(subset=["age"])
```

Replace

```
df.replace("?", np.nan)
df["col"].replace({0: np.nan})
```

Rename

```
df.rename(columns={"old": "new"}, inplace=True)
df.rename(index={0: "row0"})
```

Apply (Use Carefully)

Row-wise:

```
df.apply(lambda row: row["a"] + row["b"], axis=1)
```

Column-wise:

```
df.apply(np.mean)
```

⚠ Slow if heavy logic.

Vectorized preferred:

```
df["a"] + df["b"]
```

3 Filtering

Boolean Indexing

```
df[df["age"] > 25]
df[(df["age"] > 20) & (df["income"] < 3000)]
```

OR:

```
df[(df["a"] > 1) | (df["b"] < 5)]
```

query (Cleaner Syntax)

```
df.query("age > 20 and income < 3000")
```

Using variables:

```
threshold = 25
df.query("age > @threshold")
```

Sorting

```
df.sort_values("age")
df.sort_values(["age", "income"], ascending=[True, False])
df.sort_index()
```

4 Groupby (Feature Engineering Core)

Basic Aggregation

```
df.groupby("category")["income"].mean()
```

Multiple:

```
df.groupby("cat").agg({
    "income": ["mean", "sum"],
    "age": "max"
})
```

Named:

```
df.groupby("cat").agg(
    avg_income=("income", "mean"),
    max_age=("age", "max")
)
```

Multi-index Result

```
g = df.groupby(["cat", "region"])["income"].mean()
g.reset_index()
```

Flatten:

```
g.columns = ["_".join(col) for col in g.columns]
```

Transform (Return Same Shape)

```
df["group_mean"] =
df.groupby("cat")["income"].transform("mean")
```

Pivot Table

```
pd.pivot_table(df,
    values="income",
    index="cat",
    columns="region",
    aggfunc="mean")
```

Equivalent:

```
df.pivot_table(...)
```

5 Feature Engineering

Encoding Categories

Label Encoding

```
df["cat"] = df["cat"].astype("category")
df["cat_code"] = df["cat"].cat.codes
```

One-Hot Encoding

```
pd.get_dummies(df, columns=["cat"])
```

Drop first:

```
pd.get_dummies(df, drop_first=True)
```

Binning (Discretization)

```
pd.cut(df["age"], bins=5)
pd.qcut(df["income"], q=4)
```

Custom:

```
pd.cut(df["age"], bins=[0,18,30,50,100])
```

Datetime Features

```
df["date"] = pd.to_datetime(df["date"])
```

Extract:

```
df["year"] = df["date"].dt.year
df["month"] = df["date"].dt.month
df["dayofweek"] = df["date"].dt.dayofweek
```

Time diff:

```
df["delta"] = df["date"] - df["date"].min()
```

Rolling (Time Series Prep)

```
df["rolling_mean"] = df["value"].rolling(7).mean()
df["expanding"] = df["value"].expanding().mean()
```

Shift (Lag Features)

```
df["lag1"] = df["value"].shift(1)
```

6 Performance Tricks

Vectorization > Apply > Loop

Bad:

```
for i,row in df.iterrows():
    ...
```

Better:

```
df["c"] = df["a"] + df["b"]
```

Avoid:

```
df.apply(...)
```

Unless necessary.

Use astype for Memory

```
df["age"] = df["age"].astype("int32")
df["cat"] = df["cat"].astype("category")
```

Check memory:

```
df.memory_usage(deep=True)
```

Efficient Filtering

Convert strings to category:

```
df["cat"] = df["cat"].astype("category")
```

Avoid Chained Assignment

Bad:

```
df[df["a"] > 0]["b"] = 1
```

Correct:

```
df.loc[df["a"] > 0, "b"] = 1
```

Read Large CSV Efficiently

```
pd.read_csv("big.csv", chunksize=10000)
```

Select columns:

```
pd.read_csv("big.csv", usecols=["a", "b"])
```

ML Workflow Pattern in Pandas

```
df = pd.read_csv("data.csv")
```

```
df = df.dropna(subset=["target"])
df["cat"] = df["cat"].astype("category")
```

```
df = pd.get_dummies(df, drop_first=True)
```

```
X = df.drop("target", axis=1).values
y = df["target"].values
```

Mastery Outcome

If you master this page:

- You clean messy datasets

- You engineer features fast
 - You avoid performance traps
 - You prepare data for sklearn / torch
 - You understand real-world data wrangling
-

PAGE 5 — Visualization & EDA

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

1 Matplotlib (Low-Level Control)

Basic Setup

```
plt.figure(figsize=(6,4))
plt.plot([1,2,3], [1,4,9])
plt.show()
```

Add labels:

```
plt.xlabel("x")
plt.ylabel("y")
plt.title("Title")
plt.legend()
```

Grid:

```
plt.grid(True)
```

Line Plot (Time Series)

```
plt.plot(df["date"], df["value"], label="value")
plt.xticks(rotation=45)
plt.tight_layout()
```

Multiple lines:

```
plt.plot(x, y1, label="y1")
plt.plot(x, y2, label="y2")
plt.legend()
```

Scatter (Feature vs Target)

```
plt.scatter(df["age"], df["income"], alpha=0.5)
```

Color by class:

```
plt.scatter(X[:,0], X[:,1], c=y, cmap="viridis")
plt.colorbar()
```

Histogram (Distribution Check)

```
plt.hist(df["age"], bins=30)
```

Density:

```
plt.hist(df["age"], bins=30, density=True)
```

Subplots (Compare Many Features)

```
fig, ax = plt.subplots(2,2, figsize=(8,6))
ax[0,0].hist(df["age"])
ax[0,1].hist(df["income"])
plt.tight_layout()
```

Loop:

```
fig, axes = plt.subplots(3,3, figsize=(8,8))
for i, col in enumerate(df.columns[:9]):
    axes[i//3, i%3].hist(df[col])
```

Styles

```
plt.style.use("ggplot")
```

List:

```
plt.style.available
```

2 Seaborn (High-Level EDA)

```
sns.set(style="whitegrid")
```

Heatmap (Correlation)

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=False, cmap="coolwarm")
```

With annotations:

```
sns.heatmap(corr, annot=True, fmt=".2f")
```

Pairplot (Quick Feature Exploration)

```
sns.pairplot(df, hue="target")
```

Numeric only:

```
sns.pairplot(df.select_dtypes("number"))
```

⚠ Slow for large data.

Distribution Plots

Histogram + KDE:

```
sns.histplot(df["age"], kde=True)
```

Boxplot:

```
sns.boxplot(x="target", y="income", data=df)
```

Violin:

```
sns.violinplot(x="target", y="income", data=df)
```

Scatter with Regression Line

```
sns.regplot(x="age", y="income", data=df)
```

Grouped:

```
sns.lmplot(x="age", y="income", hue="target", data=df)
```

Count Plot (Categorical Balance)

```
sns.countplot(x="target", data=df)
```

3 EDA CHECKLIST (DO THIS EVERY PROJECT)

Target Distribution

Classification:

```
df["target"].value_counts()  
sns.countplot(x="target", data=df)
```

Regression:

```
sns.histplot(df["target"], kde=True)
```

Check imbalance.

Missing Pattern

```
df.isna().sum().sort_values(ascending=False)
```

Visual:

```
sns.heatmap(df.isna(), cbar=False)
```

Feature missing vs target:

```
df.groupby(df["feature"].isna())["target"].mean()
```



Feature vs Target

Numeric:

```
sns.scatterplot(x="feature", y="target", data=df)
```

Boxplot by class:

```
sns.boxplot(x="target", y="feature", data=df)
```

Group mean:

```
df.groupby("target")["feature"].mean()
```



Outliers

Z-score:

```
z = (df["feature"] - df["feature"].mean()) /  
df["feature"].std()  
df[np.abs(z) > 3]
```

IQR:

```
q1 = df["feature"].quantile(0.25)  
q3 = df["feature"].quantile(0.75)  
iqr = q3 - q1  
  
df[(df["feature"] < q1 - 1.5*iqr) |  
   (df["feature"] > q3 + 1.5*iqr)]
```

Visual:

```
sns.boxplot(y=df["feature"])
```



Correlation

```
corr = df.corr(numeric_only=True)  
sns.heatmap(corr)
```

Highly correlated features:

```
corr[abs(corr) > 0.9]
```

Drop multicollinearity:

```
upper = corr.where(np.triu(np.ones(corr.shape),  
k=1).astype(bool))  
to_drop = [c for c in upper.columns if any(upper[c] >  
0.9)]  
df = df.drop(columns=to_drop)
```

Leakage Detection

Red flags:

- Feature almost perfectly correlated with target
- Post-event information
- ID-like columns

Check:

```
corr["target"].sort_values(ascending=False)
```

If correlation $\approx 1 \rightarrow$ suspicious.

Distribution Shift Check (Train vs Test)

```
sns.kdeplot(train["feature"], label="train")
sns.kdeplot(test["feature"], label="test")
plt.legend()
```

Feature Importance Quick Check (Tree Model)

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier()
model.fit(X, y)

pd.Series(model.feature_importances_,
          index=df.drop("target", axis=1).columns
          ).sort_values().plot(kind="barh")
```

Fast EDA Template

```
df = pd.read_csv("data.csv")

print(df.shape)
print(df.dtypes)

sns.countplot(x="target", data=df)

corr = df.corr(numeric_only=True)
sns.heatmap(corr)

for col in df.select_dtypes("number"):
    sns.histplot(df[col])
    plt.show()
```

Performance Notes

✓ Sample large data:

```
df_sample = df.sample(10000)
```

✓ Avoid pairplot on 100k rows ✓ Use `.select_dtypes()` ✓ Disable annotation on huge heatmaps

Mastery Outcome

If you master this page:

- You detect leakage early
- You understand feature relationships fast
- You diagnose imbalance
- You detect outliers properly
- You prevent training garbage models

Workflow

```
import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import GridSearchCV,
RandomizedSearchCV

from sklearn.preprocessing import StandardScaler,
MinMaxScaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.linear_model import LinearRegression,
LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier,
GradientBoostingClassifier

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, confusion_matrix,
    classification_report
)
```

1 Train / Test Split

Basic

```
X = df.drop("target", axis=1).values
y = df["target"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

Stratified (For Classification)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)
```

⚠ Always stratify if class imbalance.

2 Preprocessing

Scaling

StandardScaler (mean=0, std=1)

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

⚠ Never fit on test.

MinMaxScaler (0–1)

```
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

Encoding Categorical

```
encoder = OneHotEncoder(handle_unknown="ignore")

X_cat = encoder.fit_transform(df[["category"]])
```

Dense:

```
encoder = OneHotEncoder(sparse_output=False)
```

ColumnTransformer (Mixed Data)

```
num_cols = ["age", "income"]
cat_cols = ["city"]

preprocessor = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"),
    cat_cols)
])
```

Pipeline (CRITICAL)

```
pipe = Pipeline([
    ("prep", preprocessor),
    ("model", LogisticRegression())
])

pipe.fit(df[num_cols + cat_cols], y)
```

Predict:

```
y_pred = pipe.predict(X_test)
```

⚠ Pipeline prevents leakage.

3 Models

Regression

Linear Regression

```
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Coefficients:

```
model.coef_
model.intercept_
```

Classification

Logistic Regression

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

Probabilities:

```
model.predict_proba(X_test)
```

KNN

```
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)
```

⚠ Needs scaling.

SVM

```
model = SVC(kernel="rbf", probability=True)
model.fit(X_train, y_train)
```

Linear:

```
SVC(kernel="linear")
```

Decision Tree

```
model = DecisionTreeClassifier(max_depth=5)
model.fit(X_train, y_train)
```

Random Forest

```
model = RandomForestClassifier(
    n_estimators=200,
    max_depth=None,
    random_state=42
)
```

```
model.fit(X_train, y_train)
```

Feature importance:

```
model.feature_importances_
```

Gradient Boosting

```
model = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1
)
model.fit(X_train, y_train)
```

4 Evaluation

Predictions

```
y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[: ,1]
```

Accuracy

```
accuracy_score(y_test, y_pred)
```

Precision / Recall / F1

```
precision_score(y_test, y_pred)
recall_score(y_test, y_pred)
f1_score(y_test, y_pred)
```

Full report:

```
print(classification_report(y_test, y_pred))
```

ROC-AUC

```
roc_auc_score(y_test, y_proba)
```

⚠ Use probabilities.

Confusion Matrix

```
confusion_matrix(y_test, y_pred)
```

Normalized:

```
cm = confusion_matrix(y_test, y_pred)
cm / cm.sum(axis=1, keepdims=True)
```

Cross-Validation

```
scores = cross_val_score(  
    model, X, y,  
    cv=5,  
    scoring="f1"  
)  
  
scores.mean()
```

Stratified default for classification.

5 Hyperparameter Tuning

GridSearch

```
param_grid = {  
    "n_estimators": [100,200],  
    "max_depth": [None,5,10]  
}  
  
grid = GridSearchCV(  
    RandomForestClassifier(),  
    param_grid,  
    cv=5,  
    scoring="f1",  
    n_jobs=-1  
)  
  
grid.fit(X_train, y_train)
```

Best:

```
grid.best_params_  
grid.best_score_
```

RandomizedSearch (Faster)

```
param_dist = {  
    "n_estimators": np.arange(50,300),  
    "max_depth": [None,5,10,20]  
}  
  
search = RandomizedSearchCV(  
    RandomForestClassifier(),  
    param_dist,  
    n_iter=20,  
    cv=5,  
    random_state=42  
)  
  
search.fit(X_train, y_train)
```

Tuning with Pipeline

```
pipe = Pipeline([  
    ("scale", StandardScaler()),  
    ("model", SVC())  
)
```

```
])  
  
param_grid = {  
    "model__C": [0.1,1,10],  
    "model__kernel": ["linear","rbf"]  
}  
  
grid = GridSearchCV(pipe, param_grid, cv=5)  
grid.fit(X_train, y_train)
```

⚠ Use model__param.

💧 Complete Template

```
X = df.drop("target", axis=1)  
y = df["target"]  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, stratify=y, test_size=0.2, random_state=42  
)  
  
pipe = Pipeline([  
    ("scale", StandardScaler()),  
    ("model", RandomForestClassifier())  
)  
  
pipe.fit(X_train, y_train)  
  
y_pred = pipe.predict(X_test)  
  
print("F1:", f1_score(y_test, y_pred))
```

⚡ Practical Rules

✓ Always stratify classification ✓ Always use Pipeline ✓ Scale for KNN/SVM/LogReg ✓ Trees don't need scaling ✓ Use cross-validation ✓ Tune hyperparameters ✓ Don't peek at test set

💧 Mastery Outcome

If you master this page:

- You can build any classical ML pipeline
- You avoid leakage
- You evaluate properly
- You tune efficiently
- You understand model tradeoffs

PAGE 7 — Deep Learning with PyTorch

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
```

1 Tensors

Creation

From list:

```
x = torch.tensor([1,2,3], dtype=torch.float32)
```

Zeros / Ones:

```
torch.zeros(3,4)
torch.ones(2,2)
torch.eye(3)
```

Random:

```
torch.randn(3,4)      # normal
torch.rand(3,4)        # uniform
torch.randint(0,10,(2,3))
```

Like:

```
torch.zeros_like(x)
torch.randn_like(x)
```

Shape Ops

```
x.shape
x.view(3,4)           # reshape (contiguous)
x.reshape(3,4)
x.flatten()
x.permute(1,0)
x.unsqueeze(0)
x.squeeze()
```

Device (CPU / GPU)

```
device = torch.device("cuda" if torch.cuda.is_available()
else "cpu")

x = x.to(device)
model = model.to(device)
```

⚠ Always move both data and model.

NumPy ↔ Torch

```
x.numpy()           # CPU only
torch.from_numpy(arr)
```

Autograd

```
x = torch.tensor(2.0, requires_grad=True)

y = x**2 + 3*x
y.backward()

x.grad
```

Disable grad:

```
with torch.no_grad():
    ...
```

Detach:

```
x_det = x.detach()
```

2 Model Definition

Basic nn.Module

```
class MLP(nn.Module):
    def __init__(self, in_dim, hidden, out_dim):
        super().__init__()
        self.fc1 = nn.Linear(in_dim, hidden)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden, out_dim)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x
```

Instantiate:

```
model = MLP(10, 64, 1)
```

Sequential (Quick Prototype)

```
model = nn.Sequential(
    nn.Linear(10,64),
    nn.ReLU(),
    nn.Linear(64,1)
)
```

Use custom when:

- Multiple inputs
- Skip connections

- Complex logic

Inspect

```
print(model)
sum(p.numel() for p in model.parameters())
```

Freeze:

```
for p in model.parameters():
    p.requires_grad = False
```

3 Training Loop (Core Pattern)

Setup

```
model = MLP(10,64,1).to(device)

criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)
```

One Training Step

```
model.train()

optimizer.zero_grad()

outputs = model(X_batch)
loss = criterion(outputs, y_batch)

loss.backward()

optimizer.step()
```

⚠ Order matters.

Full Epoch Loop

```
for epoch in range(epochs):
    model.train()
    total_loss = 0

    for X_batch, y_batch in loader:
        X_batch = X_batch.to(device)
        y_batch = y_batch.to(device)

        optimizer.zero_grad()

        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)

        loss.backward()
        optimizer.step()

    total_loss += loss.item()
```

```
print(epoch, total_loss/len(loader))
```

Evaluation

```
model.eval()

with torch.no_grad():
    preds = model(X_test.to(device))
```

⚠ Always disable grad in eval.

4 Dataset & DataLoader

Custom Dataset

```
class MyDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

DataLoader

```
dataset = MyDataset(X_train, y_train)

loader = DataLoader(
    dataset,
    batch_size=32,
    shuffle=True,
    num_workers=0
)
```

Loop:

```
for xb, yb in loader:
    ...
```

Train / Val Split

```
from torch.utils.data import random_split

train_ds, val_ds = random_split(dataset, [800,200])
```

5 Saving / Loading

Save Weights


```
torch.save(model.state_dict(), "model.pt")
```

Load:

```
model = MLP(10,64,1)
model.load_state_dict(torch.load("model.pt"))
model.eval()
```

Save Full Checkpoint

```
torch.save({
    "model": model.state_dict(),
    "optimizer": optimizer.state_dict(),
    "epoch": epoch
}, "ckpt.pt")
```

Load:

```
ckpt = torch.load("ckpt.pt")

model.load_state_dict(ckpt["model"])
optimizer.load_state_dict(ckpt["optimizer"])
```

6 Debugging & Sanity Tricks

1. Overfit Small Batch

```
small_loader = DataLoader(dataset, batch_size=10)

# train until near-zero loss
```

If cannot overfit → bug.

2. Print Shapes

```
print(X.shape)
print(outputs.shape)
```

Common error:

Expected (N, C) but got (C,)

3. Check Gradients

```
for name, p in model.named_parameters():
    print(name, p.grad.abs().mean())
```

If grad = 0 → dead network.

4. Detect Exploding Gradients

```
torch.nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)
```

5. Initialization

```
def init_weights(m):
    if isinstance(m, nn.Linear):
        nn.init.xavier_uniform_(m.weight)

model.apply(init_weights)
```

6. Classification Setup

Binary:

```
criterion = nn.BCEWithLogitsLoss()
```

Output:

```
nn.Linear(hidden,1)
```

Multiclass:

```
criterion = nn.CrossEntropyLoss()
```

Output:

```
nn.Linear(hidden,num_classes)
```

⚠ Do NOT apply softmax before CrossEntropyLoss.

7. Accuracy

```
pred = torch.argmax(outputs, dim=1)
acc = (pred == y_batch).float().mean()
```

Binary:

```
pred = torch.sigmoid(outputs) > 0.5
```

⚡ Performance Tips

✓ Use pin_memory=True (GPU) ✓ Use larger batch for GPU ✓ Avoid CPU→GPU transfer inside loop ✓ Use .item() only when needed ✓
Set torch.backends.cudnn.benchmark = True

🔥 Minimal Complete Template

```
model = MLP(10,64,1).to(device)
optimizer = optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.MSELoss()
```

```
for epoch in range(10):
    model.train()
    for xb,yb in loader:
        xb,yb = xb.to(device), yb.to(device)

        optimizer.zero_grad()
        out = model(xb)
        loss = criterion(out,yb)
        loss.backward()
        optimizer.step()
```



Mastery Outcome

If you master this page:

- You can implement any deep learning model
 - You understand autograd
 - You can train on GPU
 - You can debug exploding/vanishing gradients
 - You can save & resume training
-

PAGE 8 — NLP & Transformers

```
# Core
import re
import numpy as np
# Sklearn vectorization
from sklearn.feature_extraction.text import
CountVectorizer, TfidfVectorizer
# HuggingFace
from transformers import (
    AutoTokenizer,
    AutoModel,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)
import torch
```

1 Text Preprocessing

Basic Cleaning

```
text = "Hello!!! This is NLP, version 2.0 :)"

text = text.lower()
text = re.sub(r"[^a-z0-9\s]", "", text)
```

Tokenization (Manual)

```
tokens = text.split()
```

NLTK Example

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

tokens = word_tokenize(text)

stop_words = set(stopwords.words("english"))
tokens = [t for t in tokens if t not in stop_words]

lemmatizer = WordNetLemmatizer()
tokens = [lemmatizer.lemmatize(t) for t in tokens]
```

When NOT to preprocess heavily

⚠ For transformers:

- DO NOT remove stopwords
- DO NOT lemmatize
- DO NOT lowercase (if model is cased)

Let tokenizer handle it.

2 Vectorization

Bag of Words

```
docs = ["i love ai", "ai loves python"]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(docs)

print(vectorizer.vocabulary_)
print(X.toarray())
```

TF-IDF

```
tfidf = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1,2)
)

X = tfidf.fit_transform(docs)
```

N-grams

```
CountVectorizer(ngram_range=(1,3))
```

Word Embeddings (Static)

```
# Using pretrained GloVe via gensim
from gensim.models import KeyedVectors

model = KeyedVectors.load_word2vec_format("glove.txt",
    binary=False)

vec = model["king"]
```

Sentence Embeddings (Modern)

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

emb = model.encode(["hello world"])
```

3 Transformers (HuggingFace)

Load Tokenizer

```
model_name = "bert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(model_name)
```

Tokenize

```
inputs = tokenizer(
    "Hello world",
    padding=True,
    truncation=True,
    return_tensors="pt"
)

print(inputs["input_ids"])
print(inputs["attention_mask"])
```

Load Model (Embeddings)

```
model = AutoModel.from_pretrained(model_name)

outputs = model(**inputs)

last_hidden = outputs.last_hidden_state
cls_embedding = last_hidden[:,0,:]
```

Sequence Classification

```
model =
AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)

outputs = model(**inputs)
logits = outputs.logits
```

Prediction:

```
pred = torch.argmax(logits, dim=1)
```

Inference Template

```
def predict(text):
    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding=True
    )

    with torch.no_grad():
        outputs = model(**inputs)

    probs = torch.softmax(outputs.logits, dim=1)
    return probs
```

4 Fine-Tuning Skeleton

Dataset Format

```
class TextDataset(torch.utils.data.Dataset):
    def __init__(self, texts, labels):
        self.encodings = tokenizer(
            texts,
            truncation=True,
            padding=True
        )
        self.labels = labels

    def __getitem__(self, idx):
        item = {k: torch.tensor(v[idx]) for k,v in
self.encodings.items()}
        item["labels"] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):
        return len(self.labels)
```

Trainer API (Fastest Way)

```
training_args = TrainingArguments(
    output_dir="./results",
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    evaluation_strategy="epoch",
    num_train_epochs=3,
    logging_steps=10,
    save_strategy="epoch"
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=val_dataset
)

trainer.train()
```

Manual PyTorch Loop (More Control)

```
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)

for epoch in range(3):
    model.train()
    for batch in loader:
        optimizer.zero_grad()

        outputs = model(**batch)
        loss = outputs.loss

        loss.backward()
```

```
optimizer.step()
```

Freeze Base Model

```
for param in model.base_model.parameters():  
    param.requires_grad = False
```

5 Attention Intuition (Minimal)

Transformer input:

```
[input_ids]  
[attention_mask]  
[token_type_ids]
```

Output:

```
last_hidden_state → token embeddings  
pooler_output     → CLS pooled  
logits            → classification
```

6 Prompt Engineering Basics

Zero-Shot

```
prompt = ""  
Classify sentiment:  
  
Text: I love this movie  
Sentiment:  
""
```

Few-Shot

```
prompt = ""  
Classify sentiment:  
  
Text: I hate it  
Sentiment: Negative  
  
Text: Amazing experience  
Sentiment: Positive  
  
Text: It was fine  
Sentiment:  
""
```

System Prompt (LLMs)

You are a helpful AI assistant specialized in finance.
Always answer concisely.

Instruction Format

Instruction:
Summarize the text.

Input:
<text>

Output:

Chain-of-Thought (When Needed)

Solve step by step.
Explain reasoning before final answer.

⚠ Use only when reasoning tasks required.

7 Practical NLP Workflow

1. Define task
2. Choose baseline (TF-IDF + Logistic)
3. Upgrade → Transformer
4. Fine-tune
5. Evaluate
6. Optimize inference

8 Common Pitfalls

✗ Forget truncation ✗ Batch too large → OOM ✗ Applying softmax before CrossEntropy ✗ Over-cleaning text for transformers ✗ Not using attention_mask

⚡ Performance Tips

✓ Use torch.cuda.amp.autocast() ✓
Use gradient_accumulation_steps ✓ Use smaller models first ✓
Cache tokenized dataset ✓ Use datasets library

💧 Minimal End-to-End Template

```
model_name = "bert-base-uncased"  
  
tokenizer = AutoTokenizer.from_pretrained(model_name)  
model =  
AutoModelForSequenceClassification.from_pretrained(model_  
name, num_labels=2)  
  
dataset = TextDataset(texts, labels)  
loader = torch.utils.data.DataLoader(dataset,  
batch_size=16, shuffle=True)
```

```
optimizer = torch.optim.AdamW(model.parameters(), lr=2e-5)

for epoch in range(3):
    model.train()
    for batch in loader:
        optimizer.zero_grad()
        outputs = model(**batch)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
```



Mastery Outcome

If you master this page:

- You can build NLP pipelines
 - You understand TF-IDF vs embeddings
 - You can fine-tune transformers
 - You can deploy inference
 - You understand prompting basics
-

PAGE 9 — MLOps & Deployment

1 Model Saving

Pickle (Generic Python)

```
import pickle

with open("model.pkl", "wb") as f:
    pickle.dump(model, f)

with open("model.pkl", "rb") as f:
    model = pickle.load(f)
```

⚠ Not safe for untrusted sources.

Joblib (Better for Sklearn)

```
import joblib

joblib.dump(model, "model.joblib")
model = joblib.load("model.joblib")
```

Use for:

- Large numpy arrays
- sklearn pipelines

Torch Save

Save weights only:

```
import torch

torch.save(model.state_dict(), "model.pt")
```

Load:

```
model.load_state_dict(torch.load("model.pt"))
model.eval()
```

Save Full Checkpoint

```
torch.save({
    "model": model.state_dict(),
    "optimizer": optimizer.state_dict(),
    "epoch": epoch
}, "ckpt.pt")
```

Recommended Practice

✓ Save:

- model
- config

- tokenizer
- scaler
- feature columns

2 REST API — FastAPI

Install

```
pip install fastapi uvicorn
```

Minimal Server

```
# app.py

from fastapi import FastAPI
import joblib
import numpy as np
from pydantic import BaseModel

app = FastAPI()

model = joblib.load("model.joblib")

class Request(BaseModel):
    features: list[float]

@app.post("/predict")
def predict(req: Request):
    X = np.array(req.features).reshape(1, -1)
    pred = model.predict(X)
    return {"prediction": int(pred[0])}
```

Run Server

```
uvicorn app:app --reload
```

Visit:

```
http://127.0.0.1:8000/docs
```

Automatic Swagger UI.

Torch Inference Endpoint

```
@app.post("/predict")
def predict(req: Request):
    model.eval()
    X = torch.tensor(req.features).float().unsqueeze(0)

    with torch.no_grad():
        out = model(X)
        pred = torch.argmax(out, dim=1)

    return {"prediction": int(pred.item())}
```

Async Endpoint

```
@app.post("/predict")
async def predict(req: Request):
    ...
```

Use when:

- I/O bound
- Calling external APIs

3 Docker Basics

Dockerfile

```
FROM python:3.10

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

requirements.txt

```
fastapi
uvicorn
scikit-learn
torch
joblib
```

Build Image

```
docker build -t ml-api .
```

Run Container

```
docker run -p 8000:8000 ml-api
```

Production Tip

Use:

```
uvicorn app:app --workers 4
```

Or:

```
gunicorn + uvicorn workers
```

4 Experiment Tracking — MLflow

Install

```
pip install mlflow
```

Basic Usage

```
import mlflow

mlflow.set_experiment("my-exp")

with mlflow.start_run():
    mlflow.log_param("lr", 0.001)
    mlflow.log_metric("accuracy", 0.92)
```

Log Model

Sklearn:

```
import mlflow.sklearn

mlflow.sklearn.log_model(model, "model")
```

PyTorch:

```
mlflow.pytorch.log_model(model, "model")
```

Log Artifacts

```
mlflow.log_artifact("confusion_matrix.png")
```

Start UI

```
mlflow ui
```

Visit:

```
http://127.0.0.1:5000
```

5 Model Versioning

Naming Convention

```
model_v1.pkl
model_v2.pkl
```



```
model_2026_02_19.pt
```

Better:

```
model_task_dataset_metric.pt
```

Directory Structure

```
project/
├── models/
│   ├── v1/
│   └── v2/
├── data/
└── src/
```

Best Checkpoint Pattern (Torch)

```
best_loss = float("inf")

if val_loss < best_loss:
    best_loss = val_loss
    torch.save(model.state_dict(), "best_model.pt")
```

Load Specific Version

```
model.load_state_dict(
    torch.load("models/v2/best_model.pt")
)
```

6 Production Checklist

✓ Model deterministic (set seed) ✓ Model in eval mode ✓ No gradients in inference ✓ Validate request shape ✓ Handle bad inputs ✓ Log predictions ✓ Monitor latency ✓ Monitor drift

7 Logging (Minimal)

```
import logging

logging.basicConfig(level=logging.INFO)

logging.info("Model loaded")
logging.error("Prediction failed")
```

8 Basic Monitoring Idea

Track:

- Latency
- Input distribution

- Prediction distribution
- Error rate

Save to:

- Logs
- Database
- Monitoring tool

9 Minimal End-to-End Deployment

Train → Save → Serve → Containerize

```
# train.py
joblib.dump(model, "model.joblib")
uvicorn app:app
docker build -t ml-api .
docker run -p 8000:8000 ml-api
```

Real-World Upgrade Path

Local:

- FastAPI

Small scale:

- Docker + VM

Medium:

- Kubernetes

Cloud:

- AWS SageMaker
- GCP Vertex AI
- Azure ML

⚡ Common Pitfalls

✗ Forget model.eval() ✗ Forget scaler at inference ✗ Different preprocessing in prod ✗ Large model without batching ✗ Blocking server with heavy sync calls

🔥 Mastery Outcome

If you master this page:

- You can deploy models as APIs
- You can containerize ML systems
- You can track experiments
- You understand versioning
- You can move from notebook → production

PAGE 10 — Full Workflow Blueprint

1 End-to-End ML Pipeline

1. Problem Definition

Define:

```
TASK = "binary_classification"
METRIC = "roc_auc"
CONSTRAINTS = {
    "latency_ms": 50,
    "memory_mb": 500
}
```

Checklist:

- Input?
- Output?
- Business metric?
- Offline metric?
- Real-time or batch?

2. Data Ingestion

CSV:

```
import pandas as pd

df = pd.read_csv("data.csv")
```

Database:

```
import sqlalchemy

engine = sqlalchemy.create_engine(DB_URL)
df = pd.read_sql("SELECT * FROM table", engine)
```

API:

```
import requests
data = requests.get(URL).json()
```

Validate early:

```
assert df.shape[0] > 0
assert "target" in df.columns
```

3. EDA (Fast Triage)

```
df.head()
df.info()
df.describe()
df.isna().mean()
```

Target balance:

```
df["target"].value_counts(normalize=True)
```

Correlation:

```
df.corr(numeric_only=True)["target"]
```

4. Feature Engineering

Split:

```
X = df.drop("target", axis=1)
y = df["target"]
```

Encoding:

```
X = pd.get_dummies(X, drop_first=True)
```

Scaling:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Save scaler:

```
import joblib
joblib.dump(scaler, "scaler.joblib")
```

5. Modeling

Baseline first:

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

Upgrade:

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=300)
```

6. Validation

Split:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y
)
```

Metric:

```
from sklearn.metrics import roc_auc_score

pred = model.predict_proba(X_test)[:,-1]
```

```
roc_auc_score(y_test, pred)
```

Cross-val:

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y, cv=5)
```

7. Deployment

Save:

```
joblib.dump(model, "model.joblib")
```

Serve → FastAPI Containerize → Docker Monitor → Logs + metrics

2 Common Failure Modes

Data Leakage

Wrong:

```
scaler.fit(X) # before split ❌
```

Correct:

```
scaler.fit(X_train)
```

Leak sources:

- Target encoding using full dataset
- Time-series future data
- Feature created from label

Overfitting

Symptoms:

- Train AUC = 0.99
- Test AUC = 0.72

Fix:

- Regularization
- More data
- Simpler model
- Cross-validation

Underfitting

Symptoms:

- Train AUC = 0.60
- Test AUC = 0.58

Fix:

- More features
- More complex model
- Feature engineering

Distribution Shift

Train:

```
avg_income = 50k
```

Prod:

```
avg_income = 80k
```

Detect:

```
df_train.mean()
df_prod.mean()
```

Monitor:

- Feature drift
- Prediction drift

3 Production Concerns

Latency

Measure:

```
import time

start = time.time()
model.predict(X_batch)
latency = time.time() - start
```

Optimize:

- Smaller model
- ONNX export
- Batch inference

Monitoring

Track:

```
{
  "timestamp": ...,
  "latency_ms": ...,
  "prediction": ...
}
```

Log distribution:

```
pred.mean()
```

Drift Detection (Simple)

Compare means:

```
abs(train_mean - prod_mean)
```

Advanced:

- KS test
- PSI (Population Stability Index)

Retraining Loop

Pseudo:

```
if performance < threshold:
    retrain()
    redeploy()
```

Automate with:

- Scheduled jobs
- CI/CD pipeline

4 Scaling

Batching

Bad:

```
for x in data:
    model.predict(x)
```

Good:

```
model.predict(batch)
```

Parallelism (CPU)

```
model = RandomForestClassifier(n_jobs=-1)
```

Multiprocessing:

```
from multiprocessing import Pool
```

GPU (PyTorch)

```
device = "cuda"
model.to(device)
```

Mixed precision:

```
from torch.cuda.amp import autocast
```

Async API Scaling

```
uvicorn app:app --workers 4
```

Horizontal scaling:

- Multiple containers
- Load balancer

5 Debug Checklist

Model not learning?

- Check labels

- Check scaling
- Overfit small batch
- Print gradients
- Check class imbalance

API crashing?

- Validate input schema
- Check tensor shapes
- Ensure model.eval()

Bad performance in prod?

- Compare train vs prod features
- Check preprocessing
- Check drift

6 Minimal ML Project Template

```
ml-project/
├── data/
│   ├── raw/
│   └── processed/
├── notebooks/
├── src/
│   ├── train.py
│   ├── inference.py
│   ├── features.py
│   └── utils.py
├── models/
├── app.py
├── requirements.txt
└── Dockerfile
```

7 Minimal Starter Training Script

```
def main():
    df = load_data()
    X_train, X_test, y_train, y_test = split(df)

    model = build_model()
    model.fit(X_train, y_train)

    evaluate(model, X_test, y_test)

    save(model)

if __name__ == "__main__":
```

```
main()
```

Essential Libraries List

Core:

- numpy
- pandas
- scikit-learn
- matplotlib

DL:

- torch
- transformers

MLOps:

- fastapi
- uvicorn
- mlflow
- joblib

Optional:

- xgboost
 - lightgbm
 - catboost
 - optuna
-

Golden Rules

✓ Baseline first ✓ Split before transform ✓ Log everything ✓ Save preprocessing ✓ Monitor in production ✓ Reproduce experiments ✓
Keep models simple

The Complete Mental Model

1. Define problem
2. Clean data
3. Build baseline
4. Improve features
5. Cross-validate
6. Save + deploy
7. Monitor
8. Retrain

Repeat loop.

If You Master All 10 Pages

You can:

- Build ML systems from scratch
 - Train deep learning models
 - Fine-tune transformers
 - Deploy APIs
 - Containerize services
 - Monitor production models
 - Design scalable ML systems
-

PAGE 11 — External Resources, Upgrade Path & Mastery Map

This page = **where to go next** + **what actually matters**.

1 Official Documentation (Primary Sources Only)

Rule:

If you can't find it in docs, you don't fully know it.

Core:

- Python → <https://docs.python.org>
- NumPy → <https://numpy.org/doc>
- Pandas → <https://pandas.pydata.org/docs>
- Scikit-learn → <https://scikit-learn.org/stable>
- PyTorch → <https://pytorch.org/docs>
- HuggingFace → <https://huggingface.co/docs>

Habit:

- Use search inside docs
- Read function signatures
- Read examples
- Skim API reference

2 High-Signal Courses (Execution-Oriented)

Foundations

- Fast.ai
- CS231n
- CS224n

Production ML

- Full Stack Deep Learning
- Made With ML

Systems / Data

- MIT 6.824 (Distributed Systems)
- Berkeley CS186 (Databases)

Guideline:

Take few. Finish fully. Re-implement from scratch.

3 Must-Read Papers (One Per Month)

How to read:

1. Abstract
2. Figures
3. Conclusion
4. Only then methods

Core Milestones:

- CNN (ImageNet)
- ResNet
- Attention is All You Need
- BERT
- GPT
- Diffusion Models

Goal: Be able to implement simplified version after reading.

4 Books (Minimal, Powerful)

Practical:

- Hands-On Machine Learning
- Machine Learning Engineering

Theory:

- Deep Learning (Goodfellow)

Systems:

- Designing Data-Intensive Applications

Read slowly. Take notes. Implement concepts.

5 Practice Platforms

Competitions:

- Kaggle
- DrivenData

Datasets:

- HuggingFace Datasets
- OpenML

Rule:

Finish projects. Don't collect notebooks.

6 Production Engineering Blogs

Study real-world failures.

Search engineering blogs from:

- Uber
- Netflix
- Airbnb
- Google AI
- OpenAI
- Meta AI

Look for:

- Scaling lessons
- Infra tradeoffs
- Failure postmortems
- Data pipeline design

7 Modern LLM Stack (2026-Relevant)

Learn these concepts:

- RAG
- Vector databases
- Embeddings
- LoRA / PEFT
- Quantization
- ONNX / inference engines

Tools to explore:

- FAISS
- LangChain
- vLLM
- DeepSpeed
- SentenceTransformers

Goal: Build one working RAG app end-to-end.

8 GitHub Repos to Study

Look for:

✓ Clean structure ✓ Config-driven training ✓ Reproducibility ✓ Proper logging ✓ Clear inference pipeline
Clone. Run. Modify. Break. Fix.

9 Suggested Build Projects (Portfolio-Ready)

Level 1 — Classical ML

- Churn prediction
- Credit risk model
- House price regression
- Fraud detection

Level 2 — Deep Learning

- Image classifier (CIFAR / custom)
- Text classifier
- Tabular DL model
- Time-series forecasting

Level 3 — LLM Engineering

- RAG chatbot
- PDF question answering
- Semantic search engine
- LLM-powered API

Level 4 — Production ML

- ML model behind FastAPI
- Dockerized service
- CI/CD retraining loop
- Drift monitoring script

Each project must include:

- README

- Reproducibility steps
- Metrics
- Screenshots
- Deployed demo (optional but strong)

10 Skill Ladder (Self-Assessment)

Level 1: Can train model.

Level 2: Can tune & evaluate model.

Level 3: Can deploy API.

Level 4: Can scale inference.

Level 5: Can design ML systems.

Level 6: Can read research + implement.

Level 7: Can improve performance beyond tutorials.

Target: Level 5+.

1 1 30–90–365 Day Mastery Plan

30 Days

- Master sklearn + pandas
- Build 2 complete ML projects
- Deploy 1 API

90 Days

- Build 1 deep learning project
- Fine-tune transformer
- Dockerize + monitor

365 Days

- Build 5+ serious projects
- Read 10+ papers
- Deploy 3 systems
- Contribute to open source
- Write technical blog posts

1 2 Interview Preparation (ML Roles)

Prepare:

- Bias-variance explanation
- Data leakage examples
- Evaluation metrics tradeoffs
- System design for ML
- Scaling inference
- Feature engineering reasoning

Practice whiteboarding:

- Real-time recommendation system
- Fraud detection system
- LLM-powered support bot

1 3 Hardware & Performance Deep Dive (Optional Track)

Learn:

- GPU memory math
- Batch size vs throughput
- Mixed precision
- Quantization
- Model distillation
- Distributed training basics

1 4 Daily Habits of Strong ML Engineers

✓ Read 1 blog post per week ✓ Re-implement ideas ✓ Log experiments ✓ Version datasets ✓ Clean repo structure ✓ Refactor messy code ✓ Measure everything

1 5 Minimal Professional Stack (2026 Baseline)

You should be comfortable with:

- Python
- NumPy
- Pandas
- Scikit-learn
- PyTorch
- Transformers
- FastAPI
- Docker
- Git
- MLflow

Bonus:

- SQL
- Linux CLI
- Basic cloud (AWS/GCP)

1 6 Golden Rule

Build > Consume Deploy > Notebook Measure > Guess Reproduce > Hope Ship > Perfect

Final Perspective

If you completed Pages 1–10:

You can already build real systems.

Page 11 ensures you:

- Stay current
- Level up continuously

- Avoid tutorial trap
- Think like a production engineer
- Design scalable AI systems